The Principles of Language Translation

DSA27 — Lecture 15 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 15 · Fall 2026

Where We Are

Fourteen weeks of parts. One machine.

You have built: the `Array`, recursion, the dynamic array, linked lists, **stacks**, queues, searches, sorts, **trees**, heaps, hash tables, graphs.

Today they meet in one program:

```
>>> calculate("2 * (3 + (4 - 1))")
12.0
```

Text in, a number out. How does a machine **read**?

Today

1. What a translator does: compiler, interpreter, three stages
2. Stage 1 — the **lexer**: text → tokens
3. Grammars: precedence and associativity as **shape**
4. Stage 2 — **recursive descent**: tokens → tree
5. Stage 3 — **evaluation**: a post-order walk
6. Two routes to one answer: the Week 6 stack against the tree
7. Measured; where this grows up; the course, looking back

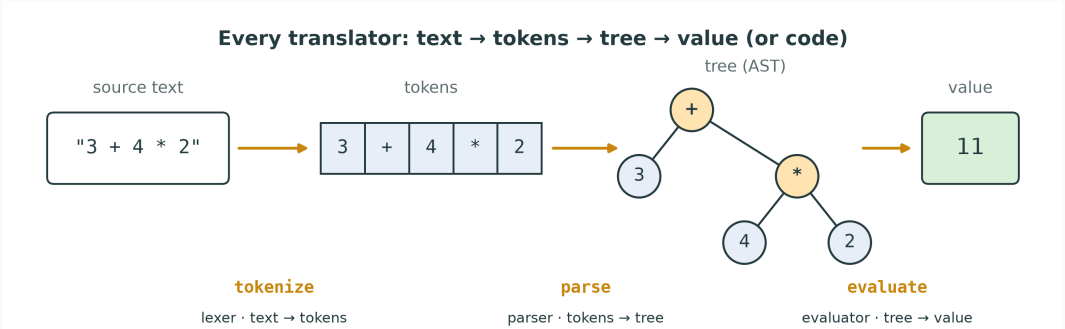
What a Translator Does

Compiler or interpreter?

	Reads the source …	… and produces	Example
Compiler	once, ahead of time	another program (machine code, bytecode)	`gcc`, `javac`
Interpreter	as it runs	the **result** directly	a shell, a calculator
Both	compiles to bytecode, then interprets it		CPython, the JVM

Our `calculate` is an interpreter. The front half is the same for both.

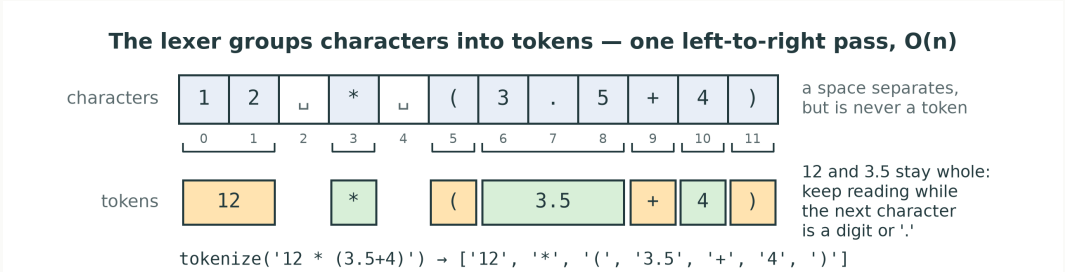
Three stages



Lexer, parser, evaluator: each stage has one job, and each can say no.

Stage 1: The Lexer

Characters into tokens



`tokenize("12 * (3.5+4)")` → `['12', '*', '(', '3.5', '+', '4', ')']`

- Whitespace **separates**, but is never a token.
- A number **keeps going** while the next character is a digit or `.`.
- Anything else is an error: `ValueError`.

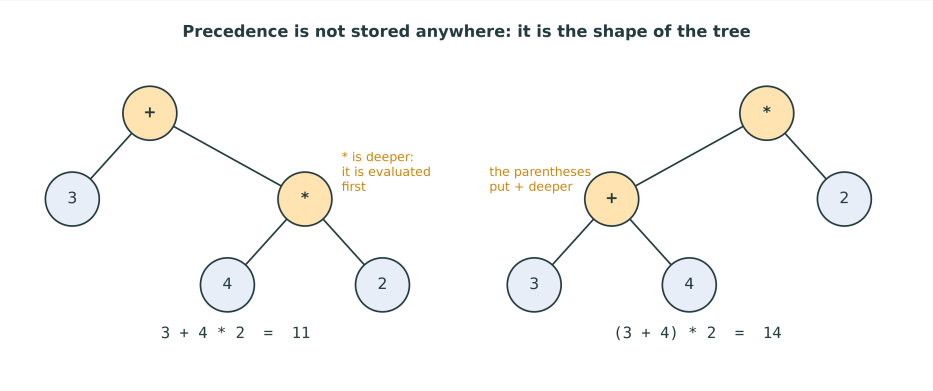
Grammars

Rules that say what is allowed

```
expr   := term (('+' | '-') term)*
term   := factor (('*' | '/') factor)*
factor := NUMBER | '(' expr ')' | '-' factor
```

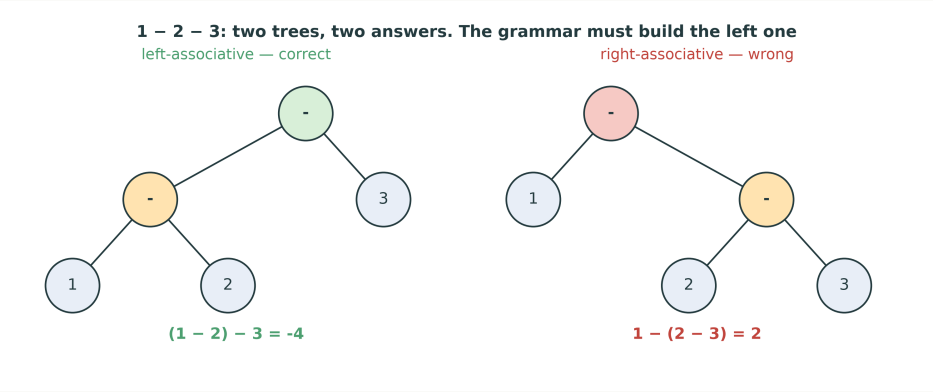
Read `:=` as “is”, `|` as “or”, `( … )*` as “zero or more times”.

Precedence is the shape of the grammar



`expr` is built from `terms`, so every * is grouped **before** any + sees it. No table of numbers — the **layering** of the rules does it.

Associativity: which way the tree leans



1 - 2 - 3 means (1 - 2) - 3 = -4. Operators of **equal** precedence group from the **left**: the tree must lean left.

Stage 2: Recursive Descent

One method per rule

Grammar	Code
a rule	a method: `expr`, `term`, `factor`
a rule named inside another	a **call**
`( … )*`	a `while` loop
`a | b | c`	`if` / `elif` on the next token
a token in quotes	check it, then `advance()`

The methods call each other **down** the grammar — and back **up** through the parentheses. Hence *recursive descent*.

expr, in full

```
def expr(self):
    node = self.term()
    while self.peek() in ("+", "-"):
        op = self.advance()
        node = BinOp(op, node, self.term())  # the old tree goes LEFT
    return node
```

`term` is the same shape, one level down: `factor` for `term`, `*` and `/` for `+` and `-`.

The loop builds a left-leaning tree

Parser.expr's loop: the tree so far becomes the LEFT child — so the tree leans left

after '1'



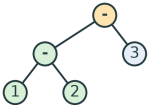

```
first:
node = self.term()
```

after '1 - 2'



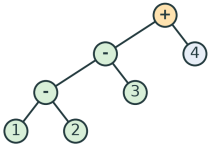

```
'-' seen:
node = BinOp('-',
     node, term())
```

after '1 - 2 - 3'




```
'-' seen:
node = BinOp('-',
     node, term())
```

after '1 - 2 - 3 + 4'




```
'+' seen:
node = BinOp('+',
     node, term())
```

`factor`: where the recursion closes

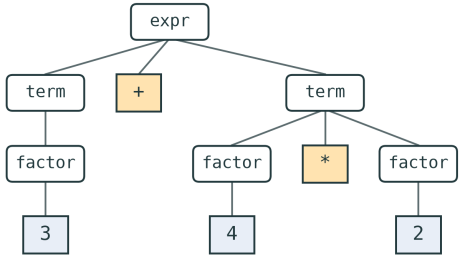
Look at the next token:

- **a number** → consume it; return `Num(float(token))`.
- `(` → consume it; `node = self.expr()`; the next token **must** be `)` — consume it; return node.
- `-` → consume it; return `BinOp("-", Num(0.0), self.factor())`.
- **anything else**, or the end → `ValueError`.

`factor` calls `expr`: the grammar's recursion, and the parser's.

The calls are the parse tree

Recursive descent on 3 + 4 * 2: the calls form a tree (the parse tree)




```
expr   := term (('+' | '-') term)*
term   := factor (('*' | '/') factor)*
factor := NUMBER | '(' expr ')' | '-' factor
```

each box is one call —
a frame on the call stack.
The deepest path is the
deepest the stack grows.

Each call is a frame on the call stack (Lecture 03). The **deepest** chain of calls is the **deepest** the stack grows.

Two grammars that look right and are not

Right recursion — `expr := term ('+' expr)?`

```
node = self.term()
if self.peek() in ("+", "-"):
    return BinOp(self.advance(), node, self.expr())
```

`1 - 2 - 3` → **2**. Wrong, silently.

Left recursion — `expr := expr '+' term | term`

```
left = self.expr()      # before reading any token
```

`RecursionError`, on every input.

Errors: say no, and say where

Input	Stage	Raised
`3 $ 4`	lexer	`ValueError`
(empty text), `3 +`, `* 3`, `()`	parser	`ValueError`
`(3 + 4`, `3 + 4 )`, `3 4`	parser	`ValueError`
`1 / 0`	evaluator	`ZeroDivisionError`

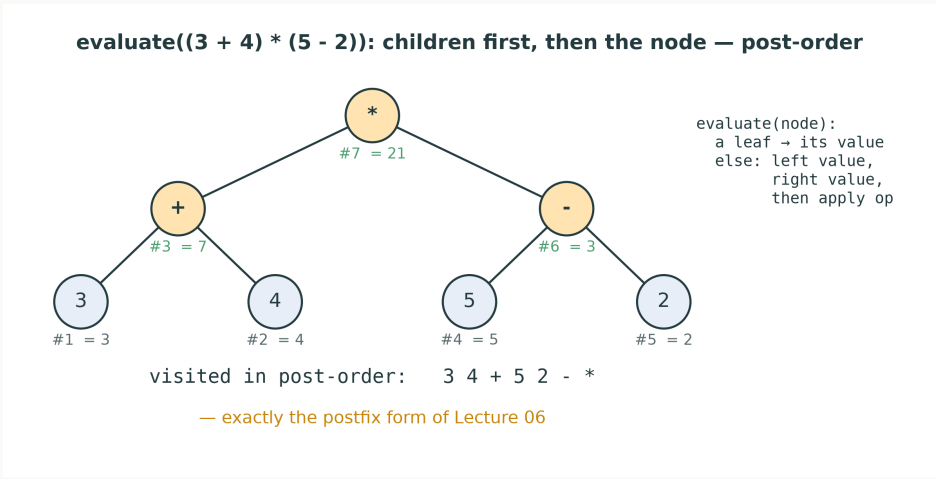
Stage 3: Walking the Tree

The AST is a Week 11 tree

- `Num(value)` — a **leaf**. `BinOp(op, left, right)` — a node with two children.
- Height = the depth of nesting. Size = the number of tokens, minus the parentheses.

```
from viz.draw import draw_tree
from dsa.translation import Parser, tokenize, to_edges
draw_tree(to_edges(Parser(tokenize("3 + 4 * 2")).parse()))
```

`evaluate` is a post-order walk



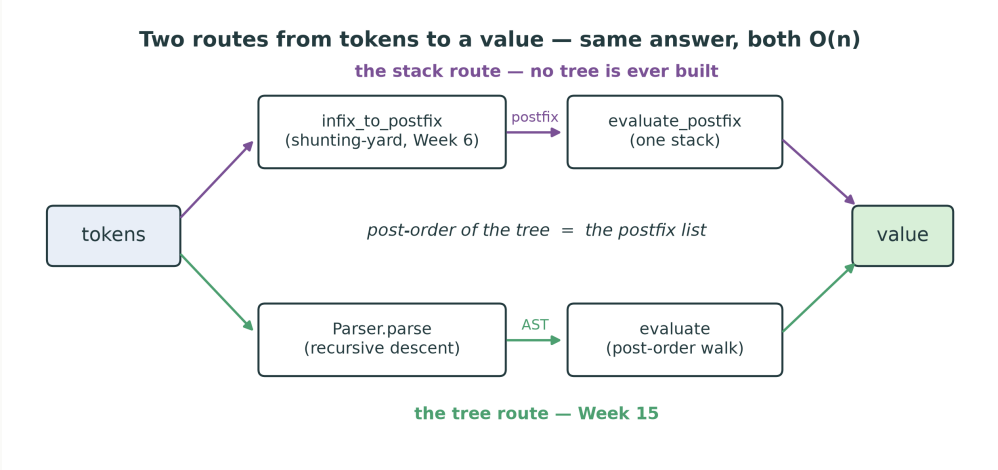
A leaf: its value. Otherwise: evaluate the **left**, evaluate the **right**, **then** apply the operator. Children first — post-order (Lecture 11).

`calculate`: the three stages, in three lines

```
tokens = tokenize(text)
tree = Parser(tokens).parse()
return evaluate(tree)
```

Two Routes, One Answer

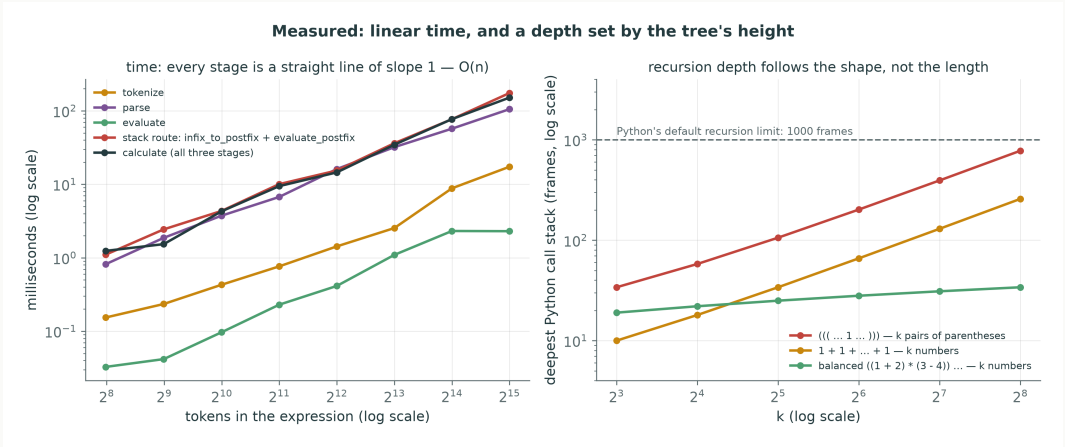
The stack route and the tree route



`test_both_routes_agree`: for `3 + 4 * 2`, `(3 + 4) * 2`, `1 - 2 - 3` and `8 / 4 / 2`, both routes give the same value.

Measured

Linear time; depth set by the shape



Where This Grows Up

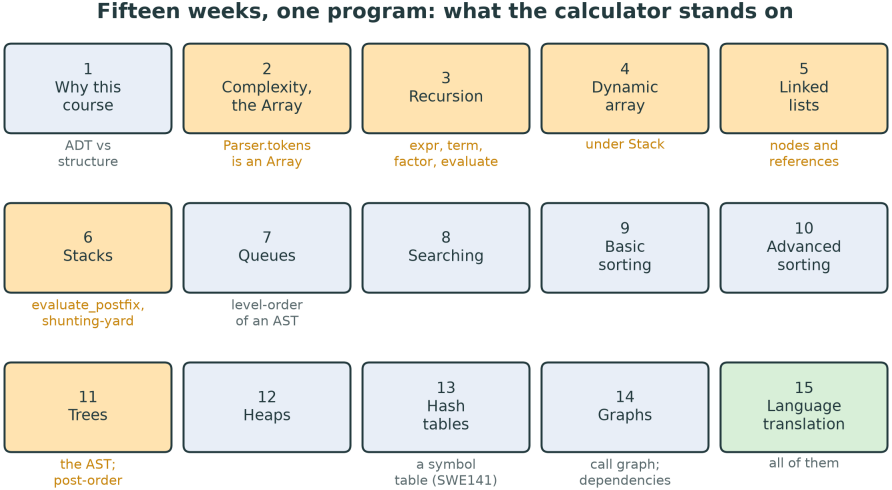
From calculator to compiler

```
>>> import ast
>>> ast.dump(ast.parse("3 + 4 * 2", mode="eval"))
Expression(body=BinOp(left=Constant(value=3),
  op=Add(), right=BinOp(left=Constant(value=4),
  op=Mult(), right=Constant(value=2))))
>>> import dis
>>> dis.dis(compile("a + b * c", "<s>", "eval"))
LOAD_NAME a;  LOAD_NAME b;  LOAD_NAME c
BINARY_OP *;  BINARY_OP +
```

Python's own tree is our tree. Its bytecode is our **postfix**.

The Course, Looking Back

Fifteen weeks, one program



What to carry out of the course

Week	The idea
1–2	An ADT is a contract; a structure keeps it at a **cost**. Measure, don't guess.
3	Recursion = a base case + progress. It **is** a stack.
4–5	Contiguous against linked: amortised $O(1)$ append vs $O(1)$ head.
6–7	Restrict access to one end or two, and the right algorithm appears.
8–10	An **invariant** (sorted) buys speed; sorting pays for it: $O(n \log n)$.
11–12	Trees: $O(h)$; balanced h = log n. A heap **is** an array.
13–14	Hashing: $O(1)$ **average**. Graphs: BFS with a queue, DFS with a stack.
15	A grammar's shape **is** its precedence; a translator uses them all.

This Week

Exercises: `dsa/translation.py`

Function	Target	The trap
`tokenize`	$O(n)$	numbers stay whole; `i < len(text)` in the inner loop; unknown character
`Parser.expr`, `term`	$O(n)$ overall	a **loop**; the old tree goes on the **left**
`Parser.factor`		consume the `)`; unary minus calls `factor`; `None` at the end
`Parser.parse`		empty input; tokens left over
`evaluate`	$O(n)$	children **first**; unknown operator
`calculate`	$O(n)$	three lines

```
pytest tests/test_translation.py -v
```

Homework 15 — before the final exam

1. **Implement** `dsa/translation.py` until all 55 tests pass.
2. **Draw** the AST, by hand, of `8 / 4 / 2`, `2 * (3 + 4) - 5` and `-(1 - 2) * 3`. Then check with `draw_tree(to_edges(...))`.
3. **Trace** the calls of `expr`, `term` and `factor` on `(1 + 2) * 3`, as an indented list — one line per call, with the tokens it consumes.
4. **Break it.** Replace the `while` in `expr` with the right-recursive version of this lecture. Which tests fail, and why only those?

Preparing for the final

- **Format:** 30 marks multiple choice, 30 marks written — traces, drawings, complexity, code on paper.
- **Pass rule:** $\geq$ 60% overall **and** $\geq$ 30% of the final.
- **Every week** has a question bank with worked answers. Do them **closed**, then mark yourself.
- **Draw** every structure after every operation. The exam rewards pictures.

Practise online

- **Week 15 quiz**: 22 multiple-choice questions, every answer explained
- **Written practice** on the same page: 5 questions. Write, then mark yourself against the marking points
- In Arabic: week 15

Summary

Seven things to keep

1. A translator has three stages: **lexer** (text $\rightarrow$ tokens), **parser** (tokens $\rightarrow$ tree), **evaluator** or code generator (tree $\rightarrow$ value or code).
2. A grammar in EBNF: `expr`, `term`, `factor` — one rule per precedence level.
3. **Precedence is the shape of the grammar**: the lower the rule, the tighter the operator.
4. **Recursive descent**: one method per rule; `( … )*` is a loop; parentheses recurse back to `expr`.
5. Left-associative = the old tree becomes the **left** child. Left recursion never terminates; right recursion gets `1 - 2 - 3` wrong.
6. Evaluation is a **post-order** walk, and post-order of the AST **is** postfix.
7. Time is $O(n)$; recursion depth is the tree's **height**.

Next

The final exam. Everything from Week 1 to Week 15.

Then, depending on your program: **SWE141 Software Construction** (grammars and parsing, grown up), **AI3001** (algorithm design), and every course that asks you to choose a structure and defend the choice.